تقرير تدقيق منظومة فلاتر كتالوج العطور

التاريخ: 05-11-2026
المستودع: `hossamthapitelgharib/elan-scents`
الفرع والنسخة: `main` عند `7e5e6559fa10973d76b7215e1bfa3866d392e7f0`
نطاق التدقيق: مراجعة مباشرة للمستودع المتاح، جرد الملفات والمجلدات، فحص مسار الكتالوج والفلاتر وقواعد العروض والأحجام والكارت والـ migrations والاختبارات والـ Worker/build. لم تُستخدم مساعدين، ولم تُجرَ كتابة في قاعدة Supabase الحية.

1. خلاصة تنفيذية

المشكلة الأساسية ليست في أسماء الفلاتر وحدها، بل في خلط ثلاثة مفاهيم داخل نموذج واحد في الواجهة:

1. **العرض الأساسي للمنصة:** أرخص عرض مؤهل بين متاجر منصة Élan Scents فقط لنفس `product_size_id`.
2. **فلتر متجر محدد:** فلتر ملكية للعرض الأساسي؛ لا يحسب سعراً بديلًا داخل المتجر.
3. **مجموعة العروض الكاملة:** كل عروض الحجم داخل متاجر المنصة، وتستخدم لحساب العرض الأساسي الوحيد والتحقق منه.

قاعدة البيانات تطبق مفهوم الأرخص بين متاجر المنصة بصورة سليمة مبدئيًا عبر `private.fn_cheapest_offers()` و`public.cheapest_offers`. لكن الواجهة العامة تقرأ `cheapest_offers` فقط في `p.offers`، ثم تستخدم `p.offers` مرة أخرى لتصفية المتجر. القاعدة التجارية المصححة هي أن هذا العرض الواحد يظهر في كل السكاشن والفلاتر، وعند اختيار متجر لا يظهر العطر إلا إذا كان المتجر المحدد صاحب هذا العرض الأساسي، مع بقاء السعر المعروض هو السعر نفسه.

يوجد حل جزئي داخل `design-layer.js`: عند وجود فلتر متجر يستخدم `p.allOffers` ثم يختار أرخص عرض داخل المتجر. هذا لا يطابق القاعدة المصححة، لأنه قد يعرض سعراً ثانيًا لنفس العطر/الحجم. المطلوب استخدام `globalOffer` المحسوب بين متاجر المنصة فقط، ثم اختبار أن متجره هو المتجر المحدد.

التقييم: الحالة الحالية غير متسقة وظيفيًا في فلتر المتجر والعرض والكارت، بينما فلاتر التقاطع بين الفئة/المناسبة/النوتة تعمل على مستوى مجموعات الربط في الواجهة إذا كانت السجلات نفسها مرتبطة بالاسم المتوقع. يلزم إصلاح مصدر الحقيقة أولًا، ثم إضافة اختبارات سلوكية تغطي كل التركيبات.

2. ما تمت مراجعته

تم جرد النسخة الحالية: **555 ملفًا متتبعًا**، منها 280 JavaScript و167 SQL و36 Markdown و13 HTML، إضافة إلى Workflows وملفات Cloudflare وSupabase والاختبارات والأصول.

المسارات المحورية التي تمت مراجعتها:

- `app-core.js`: تحميل الكتالوج، بناء `pc`, `list()`, `best()`، `PERFUMES()`, صفحات المنتج، الفلاتر، الكارت والطلب.
- `design-layer.js`: السيكشنات التلقائية والفلترة داخل السيكشن.

- `railway/editor-catalog.js` و `railway/editor-data.js` : فلاتر محرر السيكشنات على الجداول الفعلية.
- `texts.js` : ترتيب وأسماء الفئات والمناسبات والنوتات العربية والإنجليزية.
- `upabase/migrations/20261007223643_catalog_data_integrity_and_ranking.sql` قاعدة الأرخص العام و `public_products` .
- `supabase/schema-snapshot/tables-1.txt` و `tables-3.txt` و `functions.txt` : القيود والعلاقات والصلاحيات والـ views والدوال.
- اختبارات `editor-auto-results` , `editor-catalog` , `catalog-filters-style` , `storefront-xss` , اختبارات الكارت والطلبات والـ migrations.
- CI , `package.json` ، وعمليتا `check:migrations` و `build:cloudflare` .

3. النموذج الحالي للبيانات

3.1 الهوية الصحيحة للمنتج

النظام يعرض صفًا على مستوى **الحجم** وليس المنتج العام فقط:

- `products.id` : هوية العطر/المنتج.
- `product_sizes.id` : هوية العطر مع الحجم، وهي القيمة التي تُستخدم كـ `PERFUMES[].id` و `CART[].sid` .
- `store_products.id` : عرض متجر محدد لحجم محدد.
- `product_sizes.product_id` : يربط كل الأحجام بنفس العطر.

هذا النموذج مناسب، ويجب الحفاظ عليه. لا يجوز دمج الأحجام في صف واحد عند تنفيذ الفلتر، وإلا سيُعرض سعر حجم على حجم آخر.

3.2 الأرخص بين متاجر المنصة

في `20261007223643_catalog_data_integrity_and_ranking.sql` migration، الدالة `private.fn_cheapest_offers()` تختار `DISTINCT ON (sp.product_size_id)` ، مع شروط:

- المتجر نشط.
- `sp.is_available = true` .
- المخزون `NULL` أو أكبر من صفر.
- `sync_status = 'synced'` .
- الترتيب بالسعر تصاعديًا، ثم عمولة المتجر، ثم الأولوية.

هذا يحقق قاعدة العرض الأساسي: **أرخص عرض مؤهل لنفس الحجم بين متاجر منصة Élan Scents فقط.** ويجب أن يبقى هو مصدر السعر والمتجر في كل صفحات المنصة وبطاقة المنصة وكارت البراند، دون عرض سعر ثانٍ لنفس العطر/الحجم.

3.3 كل العروض

`app-core.js` يقرأ `store_products` كاملة ويضعها في `allOffers` حسب `product_size_id`، لكنه يضع في `PERFUMES[].offers` صفًا واحدًا من `cheapest_offers` فقط. هذا هو أصل الخلل عند استعمال `offers` لتصفية متجر.

3.4 فجوة معرف العرض بين السعر والكارت

التقرير المرفق أضاف نقطة جوهرية: تحميل `cheapest_offers` لا يعيد حاليًا `store_product_id`، بينما الواجهة تبني خريطة منفصلة من `store_products` باستخدام `(store_id, product_size_id)`. هذا يجعل السعر الظاهر ومعرف العرض المستخدم في الكارت والطلب مصدرين منفصلين، وقد يؤدي إلى ربط السعر بالعرض الخطأ.

القاعدة الجديدة: يجب أن يأتي `price` و`store_product_id` من نفس الصف الناتج، ولا يجوز استنتاج معرف العرض من خريطة منفصلة.

4. المشاكل المؤكدة

P0 — فلتر المتجر يجب أن يرشح صاحب العرض الأساسي فقط

الموضع: `app-core.js:131`.

الشرط الحالي مفهومه:

```
F.st === '' || p.offers.some(o => o.avail && o.store === F.st)
```

`p.offers` بُني من `cheapest_offers`، أي صف واحد فقط لكل حجم، وهذا هو مصدر السعر الصحيح. التصحيح الجذري ليس تغييره إلى `allOffers` ثم اختيار الأرخص داخل المتجر، بل يجب تثبيت قاعدة العرض الواحد:

- في كل الأوضاع: `globalOffer = min(eligible allOffers)` بين متاجر المنصة، وهو نفس `cheapest_offers`.
- عند وجود متجر: يمر المنتج فقط إذا كان `globalOffer.storeId === selectedStoreId`.
- السعر والمتجر المعروضان في كل الأوضاع هما `globalOffer.price` و`globalOffer.storeId`.
- لا يوجد `storeOffer.price` بديل قابل للعرض على المنصة.

P0 — الكارت يجب أن يثبت العرض الأساسي الوحيد

الموضعان: `app-core.js:66-73` و`app-core.js:89`.

الكارت يخزن `sid` فقط، ثم يستدعي `best(p)`، ويجمع العناصر حسب المتجر الأرخص بين متاجر المنصة. يجب تثبيت هذا العرض صراحةً وعدم السماح بإضافة عرض غير الأرخص. إذا أضاف العميل من فلتر متجر B، فهذا يعني أن B هو صاحب العرض الأساسي، ولذلك يستخدم الكارت عرض B نفسه وبالسعر نفسه.

المطلوب فصل:

- **تصفح عام:** الإضافة تستخدم `globalOffer.storeProductId`.
- **تصفح متجر محدد:** بعد اجتياز الفلتر تستخدم نفس `globalOffer.storeProductId`، ولا يوجد عرض متجر بديل.
- `CART` يجب أن يحمل أو يشتق بوضوح `storeProductId`، مع بقاء `product_size_id` للعرض والرجوع.
- تجميع الطلب يكون حسب `storeProductId` / `storeId` الفعلي المخزن، لا حسب إعادة حساب `best()`.
- عند تغير السعر أو التوفر، التحقق الخادمي هو الحاسم ويعيد رسالة واضحة.

`best` — P0 `()` لا يملك مفهوم سياق

الموضع: `app-core.js:119`.

`best(p)` يفرز `p.offers` فقط، ولا يستقبل متجرًا أو مجموعة عروض. هذا يجعل استدعاءات كثيرة ترجع السعر العام حتى داخل سياق يحتاج سعر متجر:

- `list()`.
- `prodView()`.
- `addToCart()`.
- `cartGroups()`.
- `cartHTML()`.
- `sendStoreOrder()`.
- تتبع view/cart الذي يسجل `storeProductId`.

يجب استبداله بمساعدين صريحين مثل `storeOwnsGlobalOffer`, `globalOffer`, `eligibleOffer`, مع عدم ترك استدعاء مجهول لـ `best()` في المسارات التجارية.

`allOffers` — P0 لا يطبق حاليًا نفس أهلية `cheapest_offers`

التقرير المرفق أكد أن قراءة `store_products` في `app-core.js` تحول الصفوف إلى عروض تحمل `avail:true` دون تطبيق كامل لشروط `sync_status = 'synced'`، والمتجر `active`، و`is_available = true`، و`stock_quantity IS NULL OR stock_quantity > 0`. لذلك قد يحتوي `allOffers` على عرض غير متاح أو منتهي المزامنة أو نفد مخزونه، وقد تستخدمه السيكشنات في الظهور.

يجب ألا تبني الواجهة أهلية البيع من `store_products` الخام. المطلوب View/RPC موحد للعروض المؤهلة داخل متاجر المنصة، يتضمن `store_product_id`، ويستخدم للتحقق من `cheapest_offers` وليس لاختيار سعر متجر بديل.

P0 — السعر الظاهر ومعرف العرض يجب أن يكونا زوجًا غير قابل للفصل

كل بطاقة أو صفحة أو كارت يجب أن يحمل نتيجة واحدة مثل:

```
product_size_id + store_product_id + store_id + price + currency
```

لا يكفي أن تعرض البطاقة السعر من `cheapest_offers` ثم تبحث لاحقًا عن `store_product_id` من `store_products`. أي اختلاف بين الاثنين يعرّض الطلب لرفض السعر أو مخزون متجر مختلف.

P1 — السيكشن التلقائي يختار عرضًا خاصًا مخالفًا للقاعدة

الموضع: `design-layer.js:72-89`.

عند `f.store` يستخدم `p.allOffers` ثم يختار أرخص عرض داخل المتجر. هذا مخالف للقاعدة المصححة، لأن السيكشن يجب أن يرشح حسب صاحب `globalOffer` فقط. كذلك:

- `manualList()` في `design-layer.js:97-98` يعيد `best(p)`، وهذا مناسب للعرض العام ويجب الحفاظ عليه.
- البطاقات الناتجة تعتمد على `pc(x)` من الواجهة، وقد تعرض السعر العام بدل العرض الذي تحقق منه الاختيار إن لم تمرر سياق العرض.
- قيمة المتجر تقارن بالاسم (`sameCatalogName`) بدل `storeId`؛ الاسم المكرر أو تغييره يسبب عدم اتساق.
- flags مثل `new/offers/master/soon` تُنسخ إلى كل عروض الحجم في `allOffers` عبر علم الحجم، وهو ما قد يكون صحيحًا لبعضها لكنه يحتاج تعريفًا صريحًا: هل العلم على الحجم أم على العرض/المتجر؟

P1 — فلاتر الجنس المعروضة لا تقرأ `products.gender`

في `texts.js`، الفئات المعروضة هي: `رجالي`, `نسائي`, `عطر يجمعنا`. لكن `app-core.js:173` لا يطلب `gender` من `public_products`. بدلًا من ذلك، يطابق الفئات من جدول `product_categories` مع أسماء `categories` في `app-core.js:181-182`.

هذا يعني أن فئة الرجال/النساء/الجنسين ليست مرتبطة حاليًا بحقل `products.gender` رغم وجوده ووجود قيد قيمه (`men`, `women`, `unisex`) في schema. إذا كانت الحقيقة التجارية هي الجنس، فالمصدر الصحيح هو `products.gender`، مع migration/validation تربط قيم الواجهة العربية بقيم enum النصية. أما `categories` فتظل للفئات التجارية الأخرى.

الوضع الحالي يسمح بتعارض مثل:

- `products.gender = men` لكن لا يوجد row في `product_categories` للفئة الرجالية: المنتج لا يظهر في فلتر رجالي.
- وجود category نسائي مع `gender = men`: المنتج قد يظهر في الاثنين.

P1 — تقاطعات الفلاتر تعتمد على أرقام ترتيب ثابتة في الواجهة

في `app-core.js:181-184` ، يتم تحويل اسم السجل إلى index داخل `D.ar.S` ، ثم تخزن `p.cats` , `p.occs` , `p.notes` كأرقام. هذا يربط البيانات بترتيب النصوص، لا بمعرّف السجل.

داخل `list()` ، التقاطع بين أكثر من فلتر هو AND صحيح على arrays (`cat` و `oc` و `nt`). لكن سلامة النتيجة مشروطة بأن:

- الاسم في DB يطابق الاسم الموجود في `texts.js` حرفيًا بعد escaping.
- ترتيب القيم لا يتغير.
- لا توجد أسماء عربية بديلة أو مسافات/همزات مختلفة.

هذا ليس مصدر حقيقة مستقرًا. يجب الاحتفاظ بالـ UUID داخليًا، وإسناد `filterKey` أو enum ثابت للعرض فقط.

P1 — الأسعار لا تراعي العملة عند المقارنة

`cheapest_offers` والدوال الأمامية تقارن `price` رقميًا فقط. إذا احتوت العروض على عملات مختلفة، فإن 100 USD سيُعتبر أرخص من 500 EGP بلا تحويل. وجود `currency` في السجلات لا يعالج المقارنة.

يلزم قبل اعتماد السعر:

- فرض عملة موحدة للمنصة، أو
- جدول أسعار صرف مؤرخ مع سياسة واضحة، أو
- عدم مقارنة عروض بعملات مختلفة داخل نفس المجموعة.

`NULL stock_quantity` — P2 صحيح كتوفر غير معروف، لكن الواجهة لا تميّزه

قاعدة البيانات تعتبر `NULL` متاحًا، وهذا متسق مع تعليق migration. الواجهة تعرض حد الكمية فقط عندما تكون القيمة غير `null` . يجب إبقاء السلوك، لكن تسمية الحالة داخليًا `stockUnknown` تمنع اعتبارها مخزونًا لا نهائيًا في أي منطق جديد.

P1 — اختلاف سياسة المخزون بين محرر الكتالوج والمنصة

كتالوج المحرر في `railway/editor-catalog.js` يستخدم `stock_quantity > 0` ، بينما `cheapest_offers` يقبل `NULL` باعتباره مخزونًا غير معروف وقابلًا للظهور. هذا يؤدي إلى اختلاف بين ما يظهر في المحرر وما يظهر في المنصة.

يجب اعتماد سياسة موحدة ومسمّاة: `out_of_stock` هو صفر، و `stock_unknown` هو NULL، و `available` هو متجر نشط وعرض متاح وsynced ومخزون مطابق للسياسة. ثم تستخدم المنصة والمحرر نفس View/RPC وشروط الأهلية.

P2 — تكرار العطر بسبب مستوى العرض على الحجم

`PERFUMES` يعمل بصف لكل `product_size_id` ، بينما الإحصاءات والترتيب على مستوى `product_id` . هذا مقبول فقط إذا كان قرار المنتج أن تظهر الأحجام كسطور قابلة للشراء. يجب توثيق القرار وعدم خلطه بين الصفحات؛ والخطة الحالية تعتمد مستوى الحجم صراحةً، مع إظهار الحجم والسعر والعرض والكارت على مستوى الحجم.

5. الفلاتر المطلوبة ونتيجة المراجعة

الجنس

المطلوب: رجالي، نسائي، للجنسين.
الحالة: ممثلة بصريًا في `texts.js`، لكن التطبيق يشتقها من categories لا من `products.gender`.
القرار المقترح: اعتماد `products.gender` كمصدر أساسي، وإضافة اختبار يمنع اختلاف gender عن filter key.

المناسبات

المطلوب: ليلي، نهاري، عمل، هدية، موعد.
الحالة: `product_occasions` مع `occasions` ثم تحويل الاسم إلى index. التقاطع مع الجنس/النوتة يعمل منطقيًا كـ AND إذا كانت الربوط سليمة.
القرار المقترح: استخدام IDs/keys ثابتة، وإضافة اختبارات كل مناسبة منفردة وتقاطع مناسبتين مع note/gender.

النوتات

المطلوب: عنبري، عود، خشبي، حمضي، هيل.
الحالة: `product_notes` مع `aromatic_notes` ثم تحويل الاسم إلى index. يوجد خلط لغوي محتمل في الطلب بين «حمشي» و«حمضي»؛ الكود الحالي يستخدم `حمضي` / `Citrus` في `texts.js`. يجب اعتماد اسم تجاري واحد وتطهير السجلات القديمة قبل الاختبار.

تقاطع الفلاتر

الحالة الحالية في `list()` هي AND بين كل الشروط، وهذا هو السلوك الصحيح. المشكلة ليست في عامل AND، بل في أن شرط العرض/المتجر والسعر قد يستبعد العطر قبل اكتمال التقاطع، وبأن بعض الحقول تعتمد على أسماء/indices غير ثابتة.

6. الحل الجذري للفئات والمناسبات والنوتات

5.1 مصدر الحقيقة الحالي وما هو السليم فيه

الجداول العلاقية نفسها مصممة بشكل صحيح:

- `product_categories(product_id, category_id)` بمفتاح مركب يمنع التكرار.
- `product_occasions(product_id, occasion_id)` بمفتاح مركب.
- `product_notes(product_id, note_id)` بمفتاح مركب.
- كل علاقة لها foreign key وindex على المعرف العلاقي.

كما أن `railway/editor-catalog.js` يطبق التقاطع بين الأبعاد المختلفة باستخدام `limits.every(...)`.
إذن الخلل الجذري ليس أن SQL لا يدعم التقاطع، بل أن الواجهة تفقد UUID وتحوله إلى index بعد ربطه باسم، وأن الجنس لا يدخل هذا المسار أصلًا.

5.2 الترحيل المطلوب لـ taxonomy

أضف إلى `occasions` , `categories` , و `aromatic_notes` مفتاحًا ثابتًا مثل `key` أو `slug` مع unique constraint، مع الإبقاء على `name` للعرض والترجمة فقط:

```
أو فئات تجارية منفصلة حسب القرار :categories: men, women, unisex
occasions: daytime, evening, work, date, gift
aromatic_notes: amber, oud, woody, citrus, cardamom
```

قبل إضافة unique constraint نفذ data audit للتكرارات والتهجئات المختلفة، خصوصًا `حمضي/حمشي`. لا تعالج الاختلاف بتغيير النص في JavaScript؛ أصلح السجل المرجعي واربط المنتجات بالـ UUID الصحيح.

الجنس لا يستنتج من `categories`: يستخدم `products.gender` مباشرة بقيم `men` , `women` , `unisex`. إذا كانت سجلات الفئات الحالية تحتوي قيمًا تمثل الجنس، تُرحّل مرة واحدة إلى `products.gender` بعد تقرير تعارض، ثم تمنع الواجهة استخدامها كمصدر بديل.

5.3 الاستعلام الموحد للتقاطع

أنشئ دالة/RPC أو طبقة استعلام واحدة تعيد `product_size_id` فقط بعد تطبيق كل القيود، ثم تربطها بالعرض الأساسي. المنطق المطلوب:

```sql
WHERE p.status = 'published'
  AND p.is_active
  AND (:gender IS NULL OR p.gender = :gender)
  AND (:category_id IS NULL OR EXISTS (
        SELECT 1 FROM product_categories pc
        WHERE pc.product_id = p.id AND pc.category_id = :category_id
      ))
  AND (:occasion_id IS NULL OR EXISTS (
        SELECT 1 FROM product_occasions po
        WHERE po.product_id = p.id AND po.occasion_id = :occasion_id
      ))
  AND (:note_id IS NULL OR EXISTS (
        SELECT 1 FROM product_notes pn
        WHERE pn.product_id = p.id AND pn.note_id = :note_id
      ))
```

هذا يضمن AND بين الأبعاد المختلفة. إذا دعم العقد أكثر من قيمة في البعد نفسه، يستخدم `EXISTS` مع `IN (...)` ويُوثق أن القيم داخل البعد OR، بينما تظل الأبعاد المختلفة AND. لا تستخدم join عاديًا متعددًا دون `EXISTS` أو `DISTINCT`

حتى لا تضاعف الصفوف.

5.4 عقد الواجهة الصحيح

يحمل الكتالوج العلاقات كما هي، ولا يحولها إلى أرقام:

```
{
  gender: 'unisex',
  categoryIds: ['uuid-category'],
  occasionIds: ['uuid-evening', 'uuid-date'],
  noteIds: ['uuid-oud'],
  productSizeId: 'uuid-size',
  globalOffer: { storeId, storeProductId, price, currency }
}
```

القائمة تعرض أسماء مترجمة من `key`، لكن قيمة الفلتر المرسلة والاستعلامية هي UUID أو enum ثابت. إذا اختفى سجل taxonomy أو لم يوجد mapping، يجب تسجيل خطأ data-quality في التطوير وعدم إسقاط المنتج بصمت.

5.5 التحقق المطلوب

لا يعتبر الحل مكتملًا حتى تثبت الاختبارات الحالات التالية:

1. منتج يطابق category وoccasion وnote يظهر في التقاطع الثلاثي.
2. منتج يطابق اثنين فقط لا يظهر.
3. منتج له مناسبتان يظهر في كل مناسبة منفردة.
4. `gender=men` لا يعتمد على وجود category رجالي.
5. `gender=men` لا يظهر في `women` أو `unisex`.
6. تغيير ترتيب أو ترجمة labels لا يغير النتائج.
7. اسم taxonomy غير مطابق يسجل data-quality ولا يحول العلاقة إلى نتيجة فارغة بلا تفسير.
8. كل نتيجة تعيد الحجم والعرض الأساسي و `store_product_id` الصحيح.

7. قواعد الظهور الصحيحة التي يجب تثبيتها

لكل `product_size_id`:

1. لا يظهر في أي وضع إلا إذا وُجد عرض مؤهل واحد على الأقل من متاجر المنصة: متجر نشط، متاح، synced، والمخزون موجب أو غير معروف.

2. العرض الأساسي هو الأقل سعراً بين كل العروض المؤهلة داخل متاجر المنصة لذلك الحجم. عند التعادل نستخدم نفس tie-breaker الموجود في DB: عمولة أعلى ثم priority أقل ثم tie-breaker ثابت إضافي مثل `store_id` .
3. عند اختيار متجر، يمر المنتج فقط إذا كان المتجر المحدد هو صاحب `globalOffer` ؛ لا نعيد حساب سعر داخله.
4. فلتر السعر في كل الأوضاع يقارن `globalOffer.price` .
5. لا يوجد سعر متجر بديل قابل للعرض.
6. فلتر الحجم يقارن `product_sizes.size_ml` لنفس `product_size_id` ، لا `size_label` فقط.
7. فلاتر الجنس/المناسبة/النوتة/البراند تُطبّق على `product_id` ، ثم يعاد إلى الأحجام المؤهلة والعروض المؤهلة.
8. تقاطع الفلاتر المختلفة AND؛ تعدد السجلات داخل نفس البعد OR، مثل عطر له مناسبتا عمل وهدية يظهر في كل منهما.
9. بطاقة المنصة العامة وكارت البراند يعرضان `globalOffer` .
10. بطاقة صفحة المتجر والسيكشن المربوط بمتجر يعرضان `globalOffer` بشرط أن يكون صاحبه المتجر المحدد.
11. الكارت يحمل العرض الذي اختاره العميل فعلياً، والخادم يعيد التحقق من `store_product_id` , السعر، العملة، الحجم، والتوفر.

8. ملاحظات الاختبارات الحالية

اختبارات `editor-catalog.test.js` جيدة نسبياً: تغطي تقاطع category/occasion/note، فلتر متجر، السعر الأساسي بين متاجر المنصة، الأحجام، pagination ورفض القيم غير الصحيحة. كما أن `editor-auto-results.test.js` يثبت أن السيكشن التلقائي يستخدم `allOffers` عند وجود store، لكن هذا يحتاج تصحيحاً ليستخدمه للتحقق من مالك العرض الأساسي لا لاستخراج سعر بديل.

لكن الاختبارات لا تثبت السلوك المطلوب في storefront العام:

- لا يوجد اختبار `list()` لمنتج global cheapest في Store A وعرض صالح في Store B.
- لا يوجد اختبار أن فلتر Store B لا يعرض المنتج إذا كان الأرخص بين متاجر المنصة هو Store A.
- لا يوجد اختبار أن الضغط على Add from Store B يضع `storeProductId` الخاص بـ B في الكارت.
- لا يوجد اختبار أن الكارت لا يعيد التجميع باستخدام `best` global`()` بعد اختيار متجر.
- لا يوجد اختبار gender مقابل `products.gender` .
- لا يوجد اختبار اختلاف اسم عربي/escaping أو تغيير ترتيب النصوص.
- لا يوجد اختبار عملات مختلفة.
- لا يوجد اختبار أن `price` و `store_product_id` في البطاقة والكارت والطلب قادمان من نفس العرض.
- لا يوجد اختبار أن العرض غير المتاح أو غير المتزامن أو المنقطع المخزون لا يدخل مجموعة العروض المؤهلة.
- لا يوجد اختبار لتغير العرض الأساسي بعد إضافة السطر إلى الكارت.

الخط الأساس الذي شُغّل محلياً: **1181 اختباراً، 1180 نجاحاً، 1 فشل.** الفشل غير متعلق بالفلاتر: `tests/live-turnstile-auth-probe.test.js` يتوقف بسبب `MODULE_NOT_FOUND` في `scripts/live-`

`turnstile-auth-probe.js`. كما نجح فحص migrations: 102 migration و32 rollback و71 استثناءً صريحًا، ونجح Cloudflare build.

9. ترتيب المخاطر

الأولوية	الخلل	الأثر
P0	فلتر/سيكشن المتجر يختار الأرخص داخل المتجر	ظهور سعر ثانٍ لنفس العطر/الحجم
P0	الكارت لا يثبت `globalOffer` صراحةً	احتمال إرسال عرض غير العرض الظاهر
P0	عدم وجود offer context موحد	اختلاف النتائج بين القائمة والصفحة والسيكشن والكارت
P1	gender غير مستخدم كمصدر حقيقة	تعارض رجالي/نسائي/جنسين
P1	الاعتماد على أسماء وindices	فشل صامت بعد تغيير اسم/ترتيب سجل
P1	`manualList` لا يحترم سياق المتجر	بطاقات سيكشن غير مطابقة للفلتر
P1	مقارنة السعر دون عملة	ترتيب/فلترة سعرية خاطئة إذا تعددت العملات
P1	اختلاف سياسة `NULL stock` بين المحرر والمنصة	نتائج مختلفة لنفس الحجم
P1	انفصال السعر عن `store_product_id`	الكارت/الطلب قد يستخدمان عرضًا مختلفًا عن السعر الظاهر
P2	خلط مستوى المنتج بمستوى الحجم في التكرار والترتيب	تكرار أو إخفاء أحجام دون سياسة معلنة
P2	غياب اختبارات storefront للتقاطع	عودة العطل مستقبلًا رغم نجاح الاختبارات الحالية

10. الخلاصة

قاعدة البيانات لديها أساس جيد: هوية الحجم منفصلة عن المنتج، ربط many-to-many للفئات والمناسبات والنوتات، view للأرخص بين متاجر المنصة، وفلتر محرر يطبق شروط المتجر والسعر على `store_products` الفعلية. العطل الجذري في طبقة العرض هو عدم تثبيت `globalOffer` كالعرض الوحيد في كل المنصة، مع وجود مسار في `design-layer.js` يعيد اختيار أرخص عرض داخل المتجر.

لا ينبغي إصلاح المشكلة بتعديل شرط واحد في `list()`. الإصلاح الصحيح هو بناء طبقة عروض مؤهلة واحدة تعيد السعر ومعه `store_product_id` من نفس الصف، ثم اشتقاق `globalOffer` بين متاجر المنصة وتمريره إلى القائمة والبطاقة وصفحة المنتج والإضافة للكارت والكارت والطلب والسيكشنات. فلتر المتجر يختبر فوز المتجر بالعرض الأساسي فقط، مع ضمان عدم عرض سعرين لنفس العطر/الحجم في أي مكان.